

PRÁCTICA 0

PROGRAMACIÓN CON ESP32, PLATFORMIO Y VISUAL STUDIO CODE

Juan Pedro Bandera Rubio (jpbandera@uma.es)

En esta asignatura puedes usar el framework de programación que quieras, pero recomendamos (y daremos soporte para) la programación en el editor [Visual Studio Code](#), empleando la extensión del entorno de desarrollo [PlatformIO](#).

En esta práctica 0 de la asignatura, aprenderás a crear y utilizar un proyecto en este entorno, con el que programarás tu ESP32-S3. También aprenderás a depurar código en tu microcontrolador.

INSTALACIÓN DE VISUAL STUDIO CODE

En los PCs del laboratorio ya está instalado, pero en tu PC de sobremesa o portátil tendrás que instalarlo (si no lo tienes ya, claro). Es muy fácil, simplemente accede a este enlace <https://code.visualstudio.com/download> y sigue los pasos lógicos.

Como verás, es un entorno ligero, y rápido. Y puedes usarlo en Windows, en Linux, o en Mac. No suele poner pegas cuando trasladas tus proyectos de un PC a otro. Estas son las primeras de las ventajas que ofrece. Tiene muchas más, que han hecho que se convierta, probablemente, en la herramienta preferida para programadores de todo tipo: desde páginas web y códigos de gestión en Python hasta compilación y depuración cruzada de microcontroladores, en Visual Studio Code (VSC) puedes hacer virtualmente de todo. En esta asignatura sólo vamos a atender a lo que necesitamos para la programación y depuración en ESP32-S3, pero es interesante que curiosees en las características y capacidades de VSC.

INSTALACIÓN DE LA EXTENSIÓN DE PLATFORMIO

De nuevo, en los PCs del laboratorio esto ya está hecho, pero en tu equipo tendrás que hacerlo.

Visual Studio Code es realmente potente gracias a sus [extensiones](#). Las hay para casi todo: programación C/C++, Python, gestión de proyectos con Git, etc etc etc.

Una de estas extensiones es PlatformIO IDE, que integra el entorno de desarrollo PlatformIO en VSC.

PlatformIO es un entorno de desarrollo específicamente pensado para trabajar con sistemas empujados (microprocesadores como Raspberry Pi o Beagle Bone, microcontroladores como ESP32). En junio de 2026 su extensión para VSC había sido instalada más de 4 millones de veces. Nosotros añadiremos algunas más.

Para instalar PlatformIO IDE, simplemente sigue los pasos descritos en esta web: <https://docs.platformio.org/en/latest/integration/ide/vscode.html>

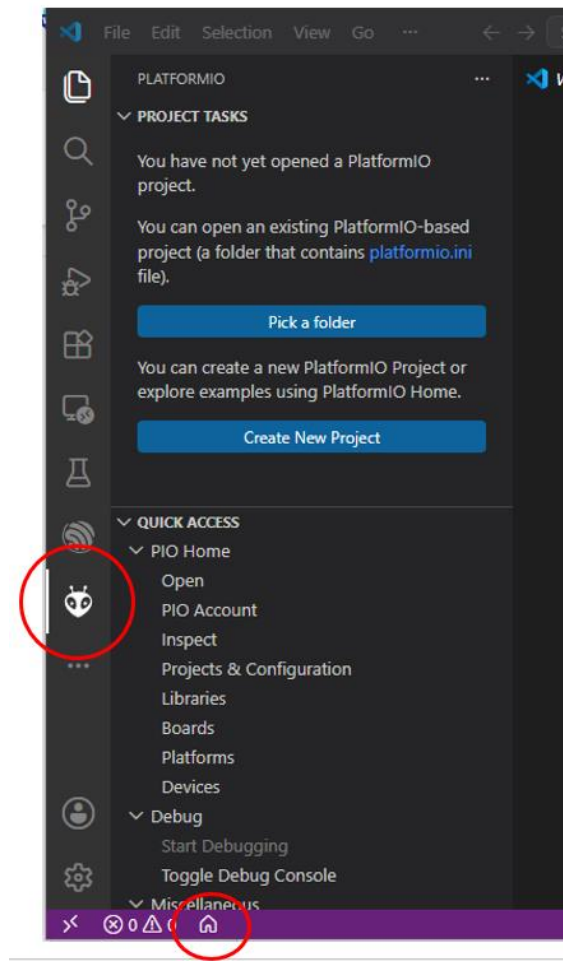
INSTALACIÓN DEL ARCHIVO DE LA PLACA, ESP32-S3-DEVKITC-1-N16R8V

PlatformIO incluye una serie de placas de ESP32 que se pueden usar por defecto en tus proyectos. Desafortunadamente, la ESP32-S3-DevKitC-1-N16R8 que usamos nosotros en la asignatura no está entre ellas, pero es posible descargar un archivo con los datos de nuestra placa de este GitHub: <https://github.com/handleexception/platform-espressif32/tree/esp32-s3-devkitc-1-n16r8v/boards>

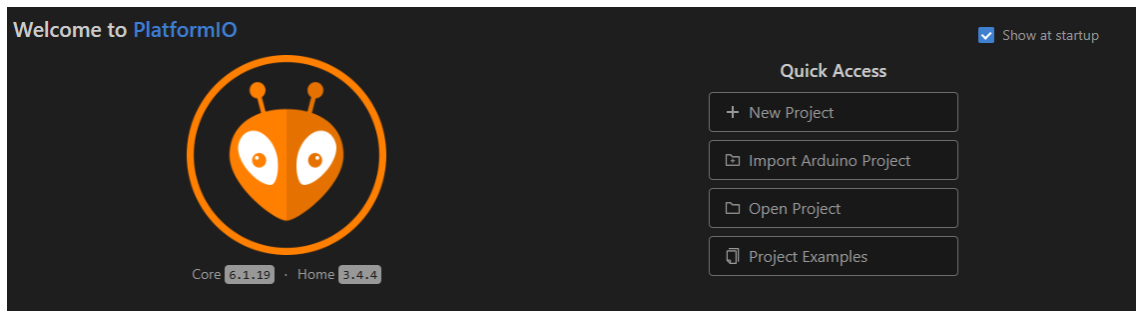
El proceso es descargar el archivo *esp32-s3-devkitc-1-n16r8v.json* de esa ubicación, y copiarlo en la carpeta *.platformio/platforms/espressif32/boards* (es una carpeta oculta que estará, como cosa normal, en tu directorio de usuario).

CREACIÓN DE UN PROYECTO PARA ESP32 EN PLATFORMIO IDE Y VSC

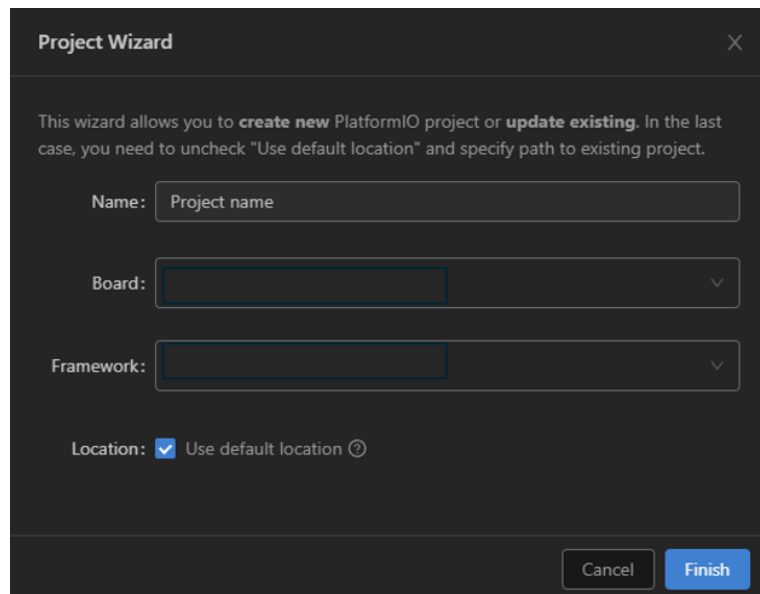
Una vez instalada la extensión de PlatformIO en VSC, aparecerá una cabeza de hormiga (el símbolo de PlatformIO) en la barra de la izquierda de VSC. Púlsala para acceder al menú que te permite crear nuevos proyectos PlatformIO o abrir una carpeta donde ya hayas creado uno. También puedes acceder al Home de la extensión pulsando la casita de abajo a la izquierda.



Una vez veas la siguiente imagen, pincha en '+ New Project' :



Y configuras el proyecto en la siguiente ventana:



Name: El nombre de tu proyecto.

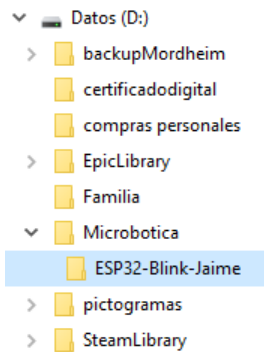
Board: Tienes que seleccionar la adecuada. Si es la que proporcionamos en Microbótica, es Espressif ESP32-S3-DevKitC-1-N16R8V (16 MB QD, 8MB PSRAM).

Framework: Puedes seleccionar el framework **Arduino** o el framework **espidf** (se suele escribir ESP-IDF). El primer framework te va a permitir realizar programas de forma más sencilla. El segundo te va a permitir que tus programas estén más optimizados, gestionar mejor el funcionamiento del ESP32, conectar mejor con el hardware, y programar más cerca de FreeRTOS.

Location: Desmarca la casilla y selecciona el directorio raíz del que quieres colgar el directorio que contendrá tu proyecto.

NOTA IMPORTANTE: En tu equipo es recomendable que crees un directorio de trabajo para tus proyectos... pero por supuesto, tu PC, tus reglas. Es sólo una recomendación.

Sin embargo, en los PCs del laboratorio, tus proyectos **DEBEN** estar siempre dentro de la carpeta que os diremos los profesores (típicamente, D:\Microbotica). Así, si Jaime quiere hacer su proyecto Blink (ESP32-Blink-Jaime), tendrá que seleccionar en **Location** el directorio D:\Microbotica. Cuando se cree el proyecto, la cosa debe quedar más o menos así.



Ten en cuenta que todo lo que dejes en los PCs del laboratorio es susceptible de ser modificado, copiado, o **borrado**. Lo ideal es que lleves tus proyectos en un pincho USB y/o que uses GitHub para tenerlos en la nube. No dejes nada en el PC del laboratorio cuando te vayas.

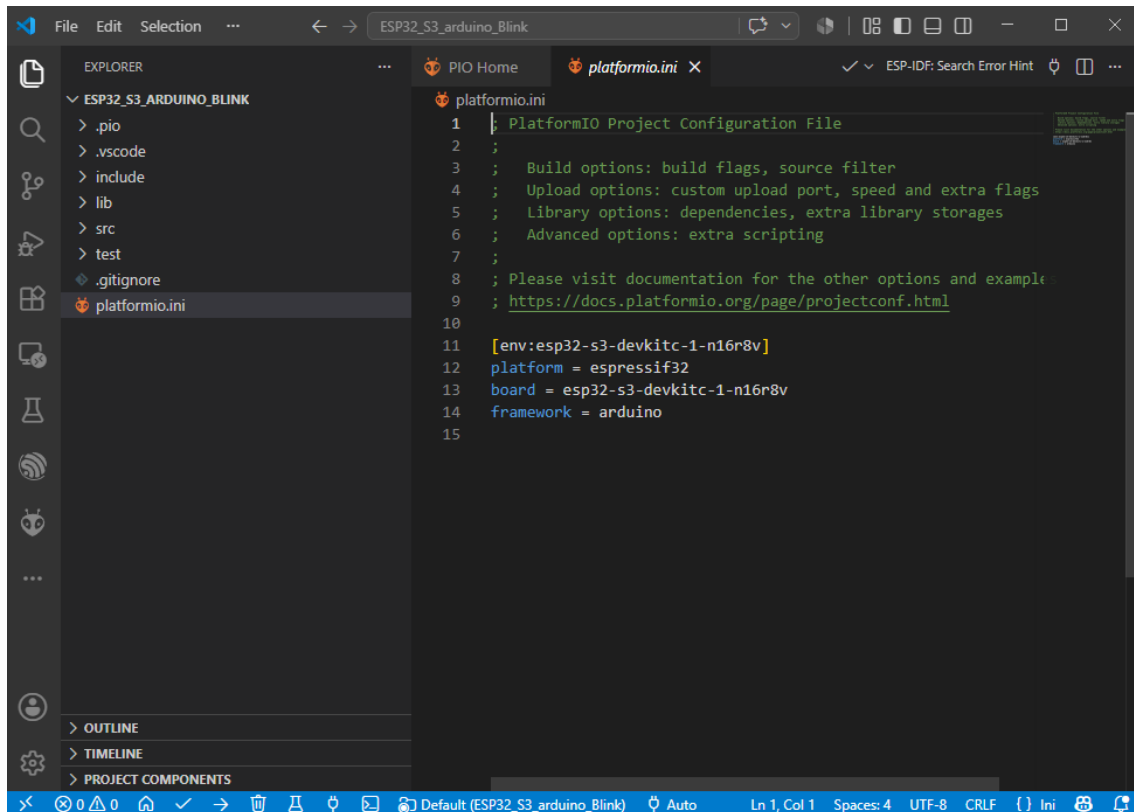
Una vez esté todo configurado dale a **Finish**, y ten paciencia. Sobre todo para los primeros proyectos que crees, el proceso de creación puede tardar bastante mientras se cargan todas las librerías y dependencias.

PRIMER PROYECTO: LED BLINK, VERSIÓN ARDUINO

El primer proyecto que crearás será uno que hará parpadear un LED con un período de 2 segundos (un segundo encendido, un segundo apagado). El LED lo conectarás, en serie a una resistencia de protección de 220Ω , al puerto GPIO que quieras.

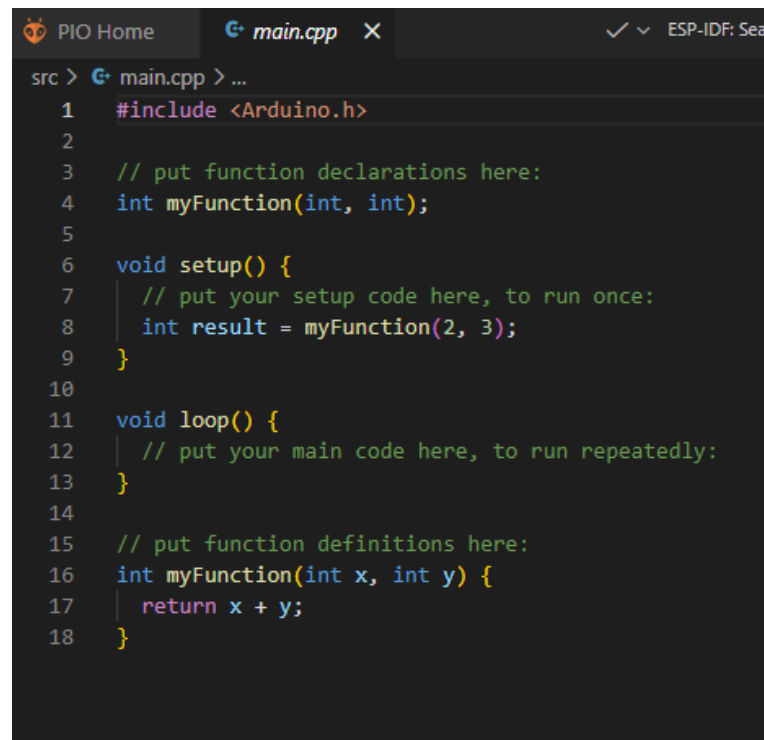
Otra opción es seleccionar como puerto de salida el GPIO4 o el GPIO5, y conectar el ESP32-S3-DevKitC-1 a la Breadboard. Con los jumpers bien colocados, tendrás conectado directamente el puerto a uno de los LED de la placa.

Montado el hardware, selecciona el Framework Arduino y crea un proyecto vacío. Cuando termine de rumiar tu ordenador, mostrará algo así:



Este es tu proyecto. Se concibe, y almacena, como un directorio, y de hecho lo abrirás y cerrarás desde el menú con las órdenes *Open Folder* y *Close Folder*. En ese directorio encuentras:

- **Directorios .pio y .vscode:** Esos no haría falta que los toques. El caso es que el segundo, el .vscode, es muy importante: contiene la información que VSC necesita para compilar, ejecutar y depurar tus programas. En otro contexto tendríamos que analizar los contenidos de este directorio. Aquí PlatformIO crea una capa de abstracción por encima que nos evita ese trabajo. De nuevo, si quieres curiosear en VSC, y en los contenidos de este directorio .vscode, ¡hazlo! Y no dudes en preguntarnos a los profesores sobre eso.
- **Directorio test:** Este tampoco lo vamos a usar en esta asignatura.
- **Directorio src:** Donde está tu programa principal. En este caso, *main.cpp*, que contiene esto:

The image shows a screenshot of the PIO Home IDE interface. At the top, there's a tab labeled 'main.cpp' with a close button. Below the tab, the code editor displays a C++ file named 'main.cpp'. The code is a standard Arduino template, starting with an include for 'Arduino.h', followed by a function declaration for 'myFunction', a 'setup()' function that calls 'myFunction', and a 'loop()' function. The 'myFunction' is defined at the bottom to return the sum of two integers. The code is as follows:

```
src > main.cpp > ...
1  #include <Arduino.h>
2
3  // put function declarations here:
4  int myFunction(int, int);
5
6  void setup() {
7      // put your setup code here, to run once:
8      int result = myFunction(2, 3);
9  }
10
11 void loop() {
12     // put your main code here, to run repeatedly:
13 }
14
15 // put function definitions here:
16 int myFunction(int x, int y) {
17     return x + y;
18 }
```

El típico proyecto vacío de Arduino, con la función de *setup()* y el bucle infinito de ejecución *loop()*.

- **Directorio include:** Donde podrás incorporar archivos de cabecera que crees para tu proyecto. Veremos más sobre esto en las siguientes prácticas.
- **Directorio lib:** Donde podrás incorporar las librerías que crees para tu proyecto. Veremos más sobre esto en las siguientes prácticas.
- **Archivo platformio.ini:** Archivo de configuración de tu proyecto. Contiene toda la información de entrada útil. Iremos viendo en sucesivas prácticas qué cosas podemos ir añadiendo. Por ahora, como vemos, indica la placa y el framework que vas a usar.

Una vez creado el proyecto, debes modificar los parámetros de entrada (en el archivo *platformio.ini*) y escribir el código del programa (en el archivo *main.cpp*). Puedes usar el proyecto *ESP32_S3_arduino_Blink*, que se suministra en el campus virtual y en el GitHub de la asignatura, como modelo.

El archivo *platformio.ini* actualizado contendrá lo siguiente:

```
[env:esp32-s3-devkitc-1-n16r8v]
platform = espressif32
board = esp32-s3-devkitc-1-n16r8v
framework = arduino

build_flags =
    -D CONFIG_BLINK_GPIO=11
monitor_speed = 115200
```

Se han añadido unas líneas, que hacen lo siguiente:

monitor_speed = 115200: configura la velocidad de la UART0 en 115200 baudios. Eso permite que, cuando monitorices el terminal serie de conexión con el microcontrolador (UART0/USB), puedas leer bien los mensajes de depuración. Esta línea habrá que añadirla, en la práctica, a TODOS los archivos *platformio.ini* en tus proyectos.

build_flags = -D CONFIG_BLINK_GPIO=11: Los build_flags son directivas para la compilación que, en este caso, se utilizan para pasar valores iniciales a tu programa. Básicamente, estas dos líneas hacen que se cree una variable inicial CONFIG_BLINK_GPIO, cuyo valor es 11. Cambia dicho valor por el puerto GPIO al que quieres conectar tu LED + resistencia serie de protección de 220Ω.

En cuanto al código en main.cpp, quedará más o menos así:


```

#include <stdio.h>
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>
#include <driver/gpio.h>
#include "sdkconfig.h"
#include <Arduino.h>

// You can change the GPIO port for the blinking LED in platformio.ini by setting CONFIG_BLINK_GPIO
// For example, to use GPIO4, set CONFIG_BLINK_GPIO=4 in platformio.ini
#define BLINK_GPIO (gpio_num_t)CONFIG_BLINK_GPIO

void blink_task(void *pvParameter)
{
    // Configure the IOMUX register for pad BLINK_GPIO
    gpio_pad_select_gpio(BLINK_GPIO);
    // Set the GPIO as a push/pull output
    gpio_set_direction(BLINK_GPIO, GPIO_MODE_OUTPUT);
    while(1) {
        // Blink off (output low)
        gpio_set_level(BLINK_GPIO, 0);
        vTaskDelay(1000 / portTICK_PERIOD_MS);
        // Blink on (output high)
        gpio_set_level(BLINK_GPIO, 1);
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
}

void setup() {
    Serial.begin(115200);
    xTaskCreate(&blink_task, "blink_task", configMINIMAL_STACK_SIZE, NULL, 5, NULL);
}

void loop() {
    Serial.println("Hello!");
    delay(1000);
}

```

Como ves, el bucle principal en este ejemplo se ha usado para mandar un mensaje “Hello!” cada segundo por el terminal serie, mientras que el parpadeo del LED se consigue creando una tarea FreeRTOS específica para ello.

Si tienes dudas sobre qué hacen las funciones, o quieres ver la API completa del framework Arduino para los GPIO del EPS32-S3, simplemente **haz click derecho sobre el nombre de una función y luego selecciona ‘Go to definition’** (o haz doble click en el nombre de función y pulsa F12). Eso abrirá el archivo *gpio.h* dentro de la estructura de PlatformIO donde se definen esas funciones.

Una vez estés satisfecho con tu código, compílalo y crea el ejecutable. Lo puedes hacer pulsando uno de los botones de PlatformIO de la barra inferior de VSC. Concretamente, el que se indica en la siguiente figura:



Debería compilar sin problemas.

Una vez creado el código, debes cargarlo en la placa ESP32-S3 (flashearlo). Eso se hace pulsando el siguiente botón:



Debería flashearlos sin problemas.

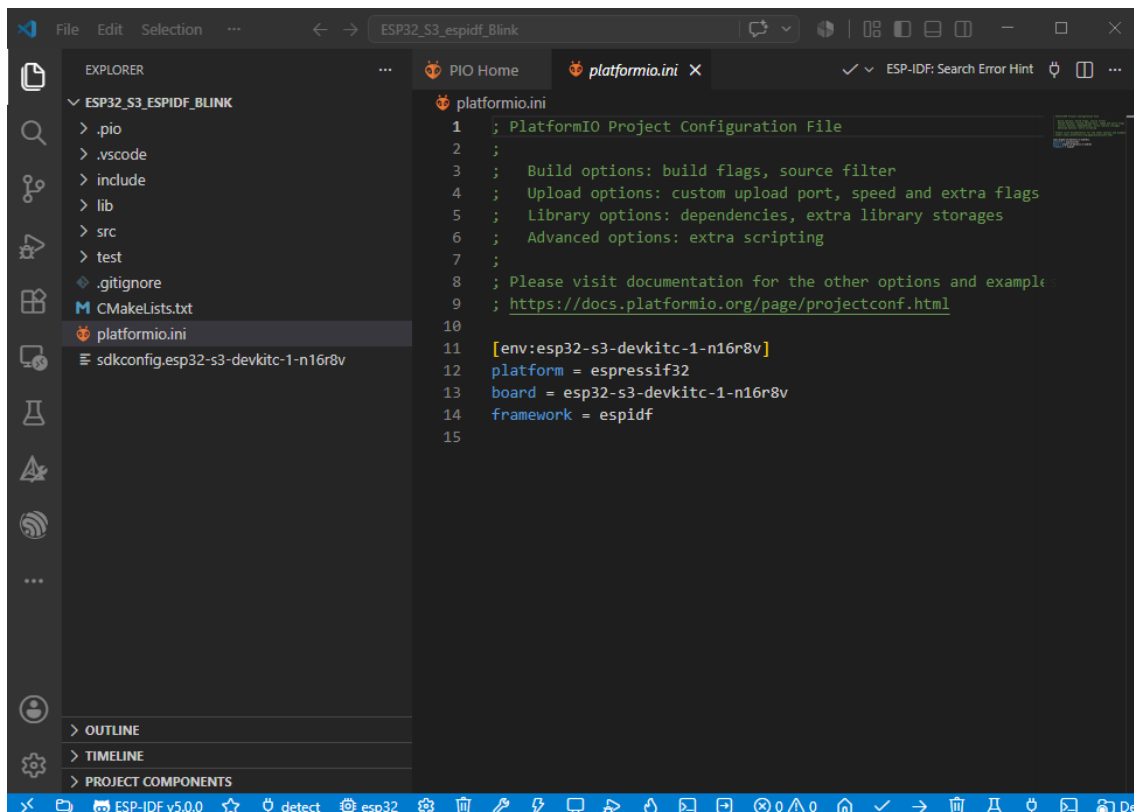
Si en algún momento quieres limpiar el proyecto (borrar todos los archivos generados al compilar) pulsa el cubo de basura. Si tienes fallos al compilar cuando te llevas código de una máquina a otra, muchas veces lo puedes arreglar simplemente limpiando el proyecto y volviéndolo a compilar.



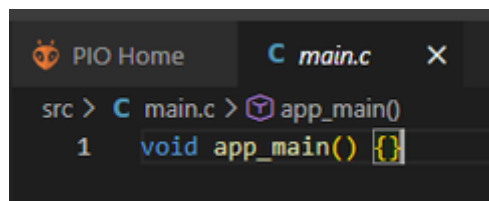
NOTA: Es buena idea limpiar el código antes de copiarlo a un pincho USB, o subirlo a la nube. El proyecto limpio ocupa muchísimo menos espacio... y de todas formas tendrás que recompilar cuando lo lleves a otra máquina.

PRIMER PROYECTO: LED BLINK, VERSIÓN ESP-IDF

Ahora vas a crear un proyecto que hace lo mismo que el anterior, pero usando el framework ESP-IDF. Para empezar, crea un nuevo proyecto, marcando como framework espidf. El resultado será algo así:



En el archivo *platformio.ini* sólo cambia el framework. En cuanto a nuestro programa (el *src/main.c*) tendrá esto:



Lo que se dice *hoja en blanco*, vaya. Pero, al igual que ocurría con el proyecto en el framework Arduino, por debajo está todo listo para que puedas empezar a desarrollar preocupándote lo mínimo por librerías, dependencias y demás.

NOTA: Fíjate que en el framework Arduino ya tienes un bucle infinito corriendo en el microcontrolador a toda pastilla, por defecto. En el framework ESP-IDF tienes un programa vacío que, si lo ejecutas, simplemente comenzará, no hará nada, y terminará.

Sigue exactamente los mismos pasos que en el proyecto de Arduino, pero partiendo del proyecto de ejemplo *ESP32_S3_espidf_Blink*. Verás que en el archivo *platformio.ini* sólo cambia el nombre del framework. En cuanto al código del programa, tiene este aspecto:

```

/* Blink Example
   Modified from original code in the Public Domain (or CC0 licensed, at your option) at
   Modified by: Juan Pedro Bandera Rubio (Dpto. Tecnología Electrónica, Universidad de M
   Date: June 2026
*/
#include <stdio.h>
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>
#include <driver/gpio.h>
#include "sdkconfig.h"

// You can change the GPIO port for the blinking LED in platformio.ini by setting CONFIG
// For example, to use GPIO4, set CONFIG_BLINK_GPIO=4 in platformio.ini
#define BLINK_GPIO (gpio_num_t)CONFIG_BLINK_GPIO

void blink_task(void *pvParameter)
{
    // Configure the IOMUX register for pad BLINK_GPIO
    esp_rom_gpio_pad_select_gpio(BLINK_GPIO);
    // Set the GPIO as a push/pull output
    gpio_set_direction(BLINK_GPIO, GPIO_MODE_OUTPUT);
    while(1) {
        // Blink off (output low)
        gpio_set_level(BLINK_GPIO, 0);
        vTaskDelay(1000 / portTICK_PERIOD_MS);
        // Blink on (output high)
        gpio_set_level(BLINK_GPIO, 1);
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
}

void hello_task(void *pvParameter)
{
    while(1) {
        printf("Hello world!\n");
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
}

void app_main()
{
    xTaskCreate(&blink_task, "blink_task", configMINIMAL_STACK_SIZE, NULL, 5, NULL);
    xTaskCreate(&hello_task, "hello_task", configMINIMAL_STACK_SIZE, NULL, 5, NULL);
}

```

Como ves, en este caso se crean dos tareas FreeRTOS, una para el parpadeo, y otra para mandar por el terminal serie el mensaje “Hello!” cada segundo, ya que en ESP-IDF no se parte de un bucle infinito vacío al programar.

Cuando compiles y cargues en el ESP32-S3, deberías ver parpadear el LED igual que en el proyecto anterior, y monitorizando el puerto serie tendrás el mismo mensaje con la misma cadencia (si no sabes monitorizar el puerto serie, es normal, lo explicamos justo en la siguiente página).

DEPURACIÓN

Programar no es sólo crear el programa. Es también depurarlo, detectar y corregir errores. El motivo principal por el que se ha escogido la placa ESP32-S3-DevKitC-1 para esta asignatura no es tanto su mayor capacidad de proceso frente a otras

alternativas de ESP32, como el hecho de que lleva un periférico de depuración USB/JTAG integrado, y fácilmente accesible.

Sin embargo, es muy normal que gran parte de la depuración se haga en *modo traza*. Sacando mensajes por el terminal serie y viendo qué va contándonos el programa conforme avanza. Como suele decirse, poniendo `printf`'s.

DEPURACIÓN: MODO TRAZA

Es la única manera que se tiene de depurar el funcionamiento de muchos micros (Arduino es bastante célebre por esto). Pero aunque tengas una placa donde puedan ponerse puntos de ruptura, inspeccionar variables, memoria, registros, y ejecutar instrucciones paso a paso... aun así, por sencillez, comodidad y eficiencia lo más normal es que uses bastante el modo traza para ir monitorizando el comportamiento del programa por el terminal serie.

Para abrir el terminal serie, simplemente pincha en el siguiente icono (si no ves nada, puede que tengas que confirmar, en la ventana de arriba del VSC, que lo que quieres ver es la salida de terminal de PlatformIO):



Si empleas el framework Arduino, tus mensajes saldrán por el terminal serie (conectado a la UART0 de tu ESP32-S3) con las variantes del `Serial.print()`. Puedes ver la API en este enlace: <https://docs.arduino.cc/language-reference/en/functions/communication/serial/print/>

Por el contrario, si empleas el framework ESP-IDF, podrías sacar los mensajes por el terminal serie con los típicos `printf` de C (<https://cplusplus.com/reference/cstdio/printf/>). No tienen nada de malo, pero hay una opción mejor. ESP-IDF te ofrece por defecto una serie de macros que te pueden ayudar a dar más información con los mensajes de traza, además de gestionar condiciones de error, entre otras cosas. Están en `esp_log.h`. Para usarlos, tienes que hacer lo siguiente:

- Incluir el archivo de cabecera:

```
#include <esp_log.h>
```

- Definir una TAG que llevarán todos los mensajes de tu programa. Por ejemplo:

```
static const char *TAG = "Blink";
```

- Emplear las macros, entre las que destacamos ahora mismo:

`ESP_LOGE`: para mensajes de error.

ESP_LOGW: para mensajes de warning.

ESP_LOGI: para mensajes de información (esto equivale a un *printf*).

ESP_ERROR_CHECK: esta no tiene tanto que ver con el modo traza, pero con ella puedes envolver una función ESP-IDF que consideres crítica, y que devuelva un *esp_err_t*. La macro comprobará lo que devuelve la función y detendrá la ejecución del programa, informando de la condición de error, si se produce alguno. Dicho de otro modo, y como se explica aquí: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/error-handling.html>

ESP_ERROR_CHECK comprueba el valor de *esp_err_t* que devuelve una función. Si *esp_err_t* NO es igual a *ESP_OK*, *ESP_ERROR_CHECK* envía por el terminal un mensaje detallado y llama a *abort()*, terminando la aplicación.

De esta forma, simplemente por cambiar

```
printf("Hello world!\n");
```

por

```
ESP_LOGI(TAG, "Hello world!");
```

tus mensajes en modo traza pasarán de este aspecto:

```
Hello World!
```

A este aspecto:

```
I (26274) Blink: Hello World!
```

Tienes el tipo de mensaje, el timestamp, el módulo que genera el mensaje, y el mensaje en sí. Si esto te gusta, te gustará aún más cuando tengas que comprobar el resultado de una función y puedas simplemente poner *ESP_ERROR_CHECK(<función>)*, en lugar de comprobar valores de vuelta con *if then else* etc etc.

Y lo que es más importante, de cara a futuro simplemente podrás cambiar el nivel de mensajes que quieres mostrar (el Log Level) para verlos todos, sólo los errores, errores+warning, ninguno en absoluto, etc. Esta es la forma profesional de generar trazas, no ya sólo para depurar tu código, sino para que puedan usarlo, verificarlo y depurarlo otras personas.

Aquí tienes la referencia completa de la *Logging Library* del ESP-IDF: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/log.html>. No es necesario que la uses, pero puede ser recomendable empezar a familiarizarse con estas prácticas de programación.

NOTA: En muchas de las prácticas de la asignatura se emplea la *Logging Library* del ESP-IDF, de forma que al recorrer las prácticas básicas ya estarás familiarizándote con estas macros.

DEPURACIÓN

En este apartado, sinceramente aún estamos trabajando para poder proporcionaros una solución unificada. Por ahora, la ejecución correcta del depurador depende de una serie de factores que pueden hacer muy fácil o difícil la tarea. Esperamos poder aportar aquí una solución única pronto (actualizaremos el documento en cuanto la tengamos). Por de pronto estos son los pasos que deberían permitirte depurar el código.

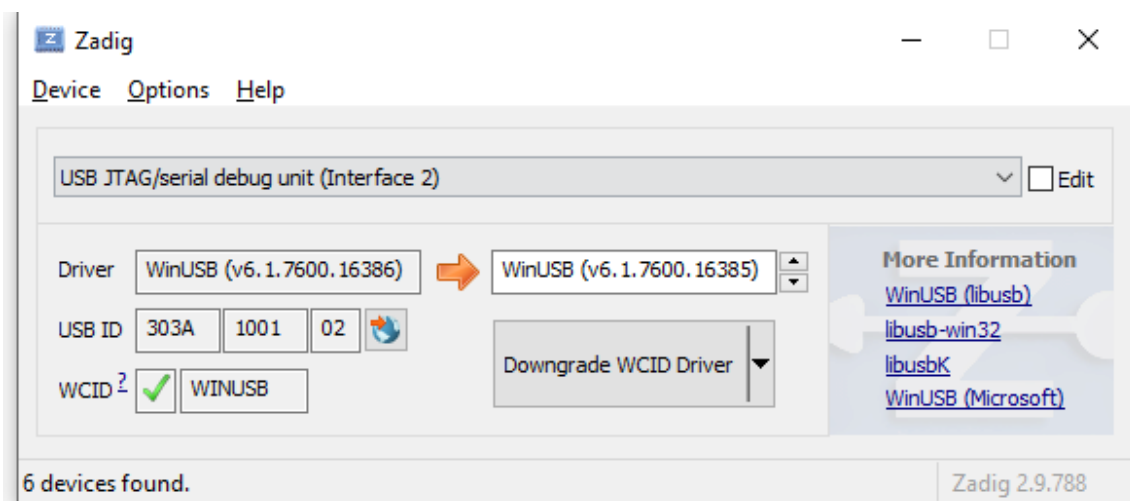
Paso previo en Windows: configuración de los drivers de los puertos USB

En Linux no es necesario hacer nada más que enchufar tu ESP32-S3 por el puerto de depuración. En el caso de Windows, sin embargo, antes de poder depurar los programas que haces en tu ESP32-S3, usando el USB/Serial JTAG Controller, es necesario realizar una serie de pasos previos. Así, suponiendo que estás trabajando en el sistema operativo Windows, tendrías que hacer lo siguiente:

1.- Conectar los dos puertos USB de tu ESP32-S3 a tu PC.

2.- Ejecutar Zadig: Zadig (<https://zadig.akeo.ie/>) es una herramienta que permite gestionar controladores para puertos USB en Windows, fácilmente. No es necesario instalar nada, simplemente te descargas el ejecutable y lo ejecutas.

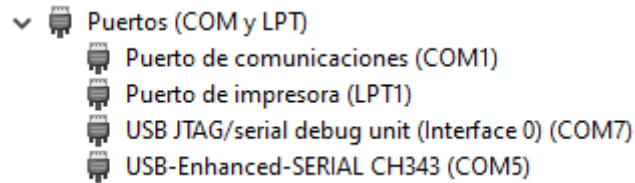
3.- Localizar el interfaz USB JTAG de tu placa: En las opciones de Zadig, si le indicas que muestre todos los dispositivos con la placa conectada, encontrarás dos *USB JTAG/serial debug unit*, uno dice 'Interface 2' y otro dice 'Interface 0'.



4.- Cambia el *USB JTAG/serial debug unit (Interface 0)*: A USB Serial (CDC).

5.- Cambia el *USB JTAG/serial debug unit (Interface 2)*: A WinUSB. Puede que ya esté configurado así.

6.- Localiza ambos puertos en el administrador de dispositivos: Deberías ver algo así.



Lo que indica, en este caso, que los puertos están bien configurados y localizados en COM7 (para carga, monitorización y depuración) y COM5 (para monitorización serie).

Depuración en el framework Arduino

Para depurar el código simplemente pulsa 'F5', 'Run->Start Debugging', o el icono de depurador en la lista a la izquierda de VSC. En Windows y Linux, en los PC que hemos empleado, no ha sido necesario hacer nada más para arrancar la sesión de depuración.

Depuración en el framework ESP-IDF

Estamos en ello. En cuanto tengamos una solución que os permita depurar en PlatformIO usando el framework espidf, en Windows y/o Linux, actualizaremos este último apartado del tutorial.